Explanation of the program is flush left.

Implementation of the program is indented.

```
## Rot13
A simple letter-substitution cipher that replaces a letter
with the letter 13 letters after it in the alphabet.

  charRot13 = (char) ->

The built-in string utility for getting character codes can be used

    charCode = char.charCodeAt(0)

If the character is in the alphabet up to 'm', then
add 13 to the character code

    charCodeRot13 = if charInRange char, 'a', 'm'
      charCode + 13

If the character is after 'm' in the alphabet, then
subtract 13 from the character code

    else if charInRange char, 'n', 'z'
      charCode - 13
    else
      charCode

Characters can be converted back using the built-in string method

    String.fromCharCode charCodeRot13

A character is in a specific range regardless of whether
it's uppercase or lowercase


  charInRange = (char, first, last) ->
    lowerCharCode = char.toLowerCase().charCodeAt(0)
    first.charCodeAt(0) <=  lowerCharCode <= last.charCodeAt(0)

Converting a string is done by converting all the characters
and joining the results

  stringRot13 = (string) ->
    (charRot13 char for char in string).join ''
```

A literate CoffeeScript file is also valid syntax for Markdown. This means that any text formatter that understands Markdown (there are many libraries for generating HTML that understand Markdown) can turn your program into a nicely formatted document.

If you prefer tests (discussed in chapter 10) and self-documenting code over comments, then you might not find the style of literate CoffeeScript to suit your taste for many programs. But even if literate CoffeeScript isn't to your taste, you still share the motivation to write programs that are well explained. That motivation also drives the invention of domain-specific languages.